```
# On branch master
# Changes not staged for commit:
#   (use "git add <file>..." to update what will be committed)
#   (use "git checkout -- <file>..." to discard changes in
working directory)
#    modified:   Oncemore.txt
no changes added to commit (use "git add" and/or "git commit
-a")
[pmu@t430 mydotfiles]$
```

The file shows up as modified. So, we need to add the file again like how Git instructs us to. Let's do that and see what happens.

```
[pmu@t430 mydotfiles]$ git add Oncemore.txt
[pmu@t430 mydotfiles]$ git status
# On branch master
# Changes to be committed:
#   (use "git reset HEAD <file>..." to unstage)
#    modified:   Oncemore.txt
[pmu@t430 mydotfiles]$
```

Has the checksum now changed?

```
[pmu@t430 mydotfiles]$ git ls-files --stage
100644 f10c977eef39357a15a18183d1a4d42b349267d 0    Oncemore.
txt
100644 b70f72952f495b2aae83f2ff1a50b5ee8d001edb 0    Readme.
txt
[pmu@t430 mydotfiles]$
```

Indeed, it has. Now, Git sees this as a completely different file...or rather, a different checksum.

Now, let's run the *git commit* command with exactly the same comment we had used earlier while committing the *Oncemore.txt* file.

```
git commit -m "Now adding the Oncemore.txt file" Oncemore.txt

[pmu@t430 mydotfiles]$ git commit -m "Now adding the Oncemore.
txt file" Oncemore.txt
[master f911d56] Now adding the Oncemore.txt file
 1 file changed, 1 insertion(+)
[pmu@t430 mydotfiles]$
```

Now, we have a new *Commit Object - f911d56*. It's time for a Git push now.

Refresh your GitHub page and you'll see that the *Oncemore.txt* file has exactly the same comment that you added just two minutes before.

Well, understand that it didn't remember the comment that you ran the first time while committing *Oncemore.txt*. This is the comment that you added just now. You could have very well added a different command.

Figure 6: *Oncemore* git push

Figure 7: Modified *oncemore*

Cloning from GitHub

Now that you've got your files up there on GitHub, what can you do with them? Cloning is one of them. Basically, when you clone a repo, you get a local copy of a Git repository so that you can fiddle around with it. Let's create a directory in which we will clone the repository.

Note: There is no need to run *git init* this time, because we are not creating a repo that we will be pushing up. We're just cloning an existing repository.

```
pmu@t430 mydotfiles]$ mkdir clonedir
[pmu@t430 mydotfiles]$ cd clonedir
[pmu@t430 clonedir]$ git clone  https://github.com/
ugrankar/mydotfiles.git
```

And what does it have?

```
[pmu@t430 clonedir]$ ls -la
total 0
drwxrwxr-x. 3 pmu pmu 23 Sep 22 22:44 .
drwxrwxr-x. 4 pmu pmu 68 Sep 22 22:44 ..
drwxrwxr-x. 3 pmu pmu 53 Sep 22 22:44 mydotfiles
[pmu@t430 clonedir]$ cd mydotfiles/
[pmu@t430 mydotfiles]$ ls
Oncemore.txt  Readme.txt
[pmu@t430 mydotfiles]$ ls -a
.  ..  .git  Oncemore.txt  Readme.txt
[pmu@t430 mydotfiles]$
```

There—you've got the same files that you had uploaded! END

By: Pritesh Ugrankar

The author is a Linux enthusiast. He has used CentOS 7 these past few weeks, and that has convinced him to move to it as a primary desktop. Please visit *www.priteshugrankar.wordpress.com* to check out a few interesting tweaks for CentOS 7.